

خطة إصلاح الأخطاء والمشاكل — على مراحل

تاريخ التقرير: 28 مايو 2026
المصدر: نتائج فحص نظام المحادثات والإشعارات
الهدف: إصلاح تدريجي آمن دون كسر أي وظيفة موجودة

فهرس المشاكل المكتشفة

#	المشكلة	النوع	الخطورة	الملفات المتأثرة
1	Push Notifications غير فعّالة بالكامل	خلل وظيفي	حرجة	pushNotifications.ts, public/, FloatingBell.tsx
2	Presence يعمل فقط في صفحة /chat	خلل وظيفي	عالية	supabaseRealtime.ts, ChatPage.tsx
3	لا يُحدّث عند إغلاق lastSeenAt المتصفح	نقص ميزة	عالية	supabaseRealtime.ts, layout.tsx
4	اختياري ← تكرار idempotencyKey محتمل للرسائل	ثغرة	عالية	MessageService.ts, api/chat/send/route.ts
5	محدود بجهاز واحد لكل Push اشتراك مستخدم	خلل	متوسطة	pushNotifications.ts
6	على Push Subscriptions تنظيف 410 فقط	نقص	متوسطة	pushNotifications.ts
7	كل تحميل (لا تخزين مؤقت للرسائل API = طلب	أداء	متوسطة	ChatPage.tsx, ChatArea.tsx
8	المخفي محدود بـ 5 Toast طابور إشعارات	فقدان بيانات	متوسطة	FloatingBell.tsx
9	Catch blocks صامتة تبتلع الأخطاء	مراقبة	متوسطة	FloatingBell.tsx, MessageService.ts
10	لكن نص DB الرسالة مشفرة في WebSocket عادي في	أمن	متوسطة	MessageService.ts, supabaseRealtime.ts
11	بعد 3 ثواني (بطيء typing_stop قليلاً)	تجربة	منخفضة	ChatArea.tsx
12	حد 2000 حرف للرسائل	قيد	منخفضة	api/chat/send/route.ts
13	عند فشل إنشاء إشعار في retry لا DB	مرونة	منخفضة	MessageService.ts

المرحلة 1: إصلاحات البنية التحتية الحرجة

الهدف: جعل الميزات الموجودة تعمل فعلياً لأول مرة
لا يمس أي كود موجود — **نوع التغيير:** إضافي (Additive)
الخطورة: منخفضة جداً — إضافة قطع مفقودة لا تعديل قائمة

1.1 Service Worker إضافة — Push Notifications إصلاح

مثبتة والكود موجود في **web-push** غير موجود. المكتبة (Service Worker) **public/sw.js** **المشكلة:** ملف **pushNotifications.ts**. Push. يستقبل أحداث الـ Service Worker ولكن لا يوجد.

الحل: إنشاء ملف **public/sw.js** في Service Worker

```
// public/sw.js
self.addEventListener("push", (event) => {
  const data = event.data?.json() ?? {}
  const { title, body, icon, badge, url, ...options } = data
  event.waitUntil(
    self.registration.showNotification(title || "إشعار", {
      body: body || "",
      icon: icon || "/icons/icon-192x192.png",
      badge: badge || "/icons/icon-96x96.png",
      url: url || "/",
      ...options,
    })
  )
})

self.addEventListener("notificationclick", (event) => {
  event.notification.close()
  const url = event.notification.data?.url || "/"
  event.waitUntil(
    clients.matchAll({ type: "window" }).then((windowClients) => {
      const existing = windowClients.find((c) => c.url === url)
      if (existing) {
        existing.focus()
      } else {
        clients.openWindow(url)
      }
    })
  )
})
```

الواردة Push **التأثير:** بدون هذا الملف، المتصفح لا يعرف كيف يتعامل مع أحداث.

1.2 للتسجيل API Route إضافة — Push Notifications إصلاح

من المتصفح Push لتسجيل اشتراك **POST /api/push/subscribe** **المشكلة:** لا يوجد endpoint

الحل: إنشاء مسار API جديد

```
src/app/api/push/subscribe/route.ts
```

```
import { NextResponse } from "next/server"
import { prisma } from "@lib/prisma"
import { verifyAuth } from "@lib/auth"

export async function POST(req: Request) {
  const session = await verifyAuth()
  if (!session) {
    return NextResponse.json({ error: "غير مصرح" }, { status: 401 })
  }

  const { endpoint, keys } = await req.json()
  if (!endpoint || !keys) {
    return NextResponse.json({ error: "بيانات غير صالحة" }, { status: 400 })
  }

  // لاستبدال الاشتراك السابق أو إضافة جديد upsert استخدام
  await prisma.pushSubscription.upsert({
    where: { endpoint_userId: { endpoint, userId: session.userId } },
    update: { p256dh: keys.p256dh, auth: keys.auth },
    create: {
      userId: session.userId,
      endpoint,
      p256dh: keys.p256dh,
      auth: keys.auth,
    },
  })

  return NextResponse.json({ success: true })
}

export async function DELETE(req: Request) {
  const session = await verifyAuth()
  if (!session) {
    return NextResponse.json({ error: "غير مصرح" }, { status: 401 })
  }

  const { endpoint } = await req.json()
  if (!endpoint) {
    return NextResponse.json({ error: "بيانات غير صالحة" }, { status: 400 })
  }

  await prisma.pushSubscription.deleteMany({
    where: { endpoint, userId: session.userId },
  })

  return NextResponse.json({ success: true })
}
```

unique ك endpoint_userId مع حقل Prisma schema في PushSubscription ملاحظة: يجب التأكد من وجود نموذج composite. يجب استخدام findFirst/delete إذا لم يكن موجوداً، يجب استخدام upsert بدلاً من.

المتصفح لا يستطيع تسجيل نفسه لتلقي الإشعارات، API التأثير: بدون هذا الـ

FloatingBell إضافة زر التفعيل في — Push Notifications إصلاح 1.3

في الواجهة (Notification Permission) المشكلة: لا يوجد زر لطلب الإذن.

Notification.permission === "default". عندما يكون FloatingBell الحل: إضافة زر "تفعيل الإشعارات" في اللوحة المنبثقة لـ

الموقع: src/components/ui/FloatingBell.tsx

التغيير: إضافة دالة وسيطة داخل المكون

```
// إضافة دالة - FloatingBell.tsx داخل
const requestPushPermission = useCallback(async () => {
  if (!("Notification" in window)) return

  const permission = await Notification.requestPermission()
  if (permission !== "granted") return

  // تسجيل Service Worker
  const registration = await navigator.serviceWorker.register("/sw.js")

  // الاشتراك في Push
  const subscription = await registration.pushManager.subscribe({
    userVisibleOnly: true,
    applicationServerKey:
      urlBase64ToUint8Array(process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY!),
  })

  // إرسال الاشتراك إلى الخادم
  await csrfFetch("/api/push/subscribe", {
    method: "POST",
    body: JSON.stringify(subscription.toJSON()),
  })
}, [])
```

JSX وإضافة زر شرطي في الـ

```
{typeof Notification !== "undefined" && Notification.permission === "default" && (
  <button onClick={requestPushPermission} className="..." >
    🔔 تفعيل الإشعارات
  </button>
)}
```

التأثير: المستخدم لا يملك وسيلة لتفعيل الإشعارات حالياً.

Layout عاماً — نقله إلى Presence جعل 1.4

المشكلة: `trackPresence()` يُستدعى فقط من `ChatPage.tsx` (السطر ~452)، (أي صفحة أخرى، (السطر ~452)). إذا كان المستخدم في أي صفحة أخرى، (السطر ~452)). لا يتم تتبع حضوره. هذا يؤثر على:

- دقة قائمة المتصلين
- خاطئ false يعطي `isUserOnline` (لأن Push Notification قرار إرسال)
- عرض حالة الاتصال في الشات

الحل:

1. `trackPresence` من `ChatPage.tsx` إلى `src/app/layout.tsx` نقل استدعاء.
2. `useSupabaseRealtime` (مع Supabase channel أو مباشرة مع) `useSupabaseRealtime` ربطه مع.
3. `cleanupPresence()` عند تسجيل الخروج (في) `cleanupPresence()` التأكد من استدعاء.

التغيير المحدد:

لتتبع الحضور `useEffect` إضافة — `src/app/layout.tsx` في:

```
// إضافة - داخل layout.tsx
useEffect(() => {
  const userStr = sessionStorage.getItem("user")
  if (!userStr) return
  const user = JSON.parse(userStr)
  if (user?.id) {
    trackPresence(user.id)
  }
  return () => cleanupPresence()
}, [])
```

`ChatPage.tsx`. ثم إزالة نفس الكود من

يظهرون كغير متصلين `/chat` التأثير: بدون هذا الإصلاح، المستخدمون خارج صفحة

عند إغلاق المتصفح `lastSeenAt` تحديث 1.5

في قاعدة البيانات. النظام يعتمد فقط على `lastSeenAt` **المشكلة:** عند إغلاق المتصفح أو التبويب، لا يتم تحديث حقل `lastSeenAt` (قد يستغرق 30-60 ثانية حتى يكتشف الخادم انقطاع الاتصال) `WebSocket` الذي يفقد الاتصال عند قطع Presence API

بسيط API عبر `lastSeenAt` يُحدَّث `layout.tsx` عالمي في `beforeunload` **الحل:** إضافة

التغيير:

1. بسيط API إنشاء.

```
src/app/api/user/ping/route.ts
```

```
import { NextResponse } from "next/server"
import { prisma } from "@lib/prisma"
import { verifyAuth } from "@lib/auth"

export async function POST(req: Request) {
  const session = await verifyAuth()
  if (!session) {
    return NextResponse.json({ error: "غير مصرح" }, { status: 401 })
  }

  await prisma.user.update({
    where: { id: session.userId },
    data: { lastSeenAt: new Date() },
  })

  return NextResponse.json({ success: true })
}
```

2. إضافة `beforeunload` في `layout.tsx`:

```
useEffect(() => {
  const handleUnload = () => {
    navigator.sendBeacon("/api/user/ping", JSON.stringify({}))
  }
  window.addEventListener("beforeunload", handleUnload)
  window.addEventListener("pagehide", handleUnload)
  return () => {
    window.removeEventListener("beforeunload", handleUnload)
    window.removeEventListener("pagehide", handleUnload)
  }
}, [])
```

يبقى عالقاً في آخر نشاط معروف، وقد يظهر المستخدم "متصلاً" لفترة بعد خروجه `lastSeenAt`، التأثير: بدون هذا الإصلاح الفعلي.

المرحلة 2: منع فقدان البيانات والتكرار

الهدف: ضمان سلامة البيانات ومنع التكرار
نوع التغيير: تعديل كود موجود + إضافات
الخطورة: متوسطة — يجب اختبار كل تغيير جيداً

تلقائياً من الخادم idempotencyKey — جعل Idempotency تعزيز 2.1

المشكلة: idempotencyKey في ال Zod schema اختياري في ال (idempotencyKey:

z.string().max(64).optional()), إذا لم يُرسله العميل (أو إذا كان هناك خطأ في الشبكة وأعاد العميل الإرسال)، يمكن أن تُنشأ رسائل مكررة.

الحل: تعديل Zod تلقائياً إذا لم يُرسل، أو جعله إجبارياً في ال idempotencyKey لإنشاء MessageService.sendMessage على مستوى الخادم schema.

الموقع: src/app/api/chat/send/route.ts + src/services/chat/MessageService.ts

API route: التغيير في

```
// تعديل Zod schema
const schema = z.object({
  receiverId: z.string().min(1),
  body: z.string().min(1).max(2000),
  replyToId: z.string().optional(),
  idempotencyKey: z.string().max(64).optional(),
})
```

MessageService: التغيير في

```
// إضافة idempotency قبل التحقق من sendMessage داخل
if (!idempotencyKey) {
  idempotencyKey = `${senderId}-${Date.now()}-${crypto.randomUUID()}`
}
```

التأثير: الرسائل المكررة ممكنة حالياً.

Push دعم أجهزة متعددة لكل مستخدم في 2.2

المشكلة: pushNotifications.ts يستخدم findUnique({ where: { userId } }) في Prisma، مما يعني أن المستخدم يمكنه امتلاك اشتراك واحد فقط. تسجيل الدخول من جهاز آخر يحل محل الاشتراك السابق.

الحل: تغيير findUnique إلى findMany لإرسال الإشعار إلى جميع الأجهزة المسجلة.

الموقع: src/lib/pushNotifications.ts

التغيير:

```
// قبل
const subscription = await prisma.pushSubscription.findUnique({
  where: { userId },
})

// بعد
```

```
const subscriptions = await prisma.pushSubscription.findMany({
  where: { userId },
})

for (const sub of subscriptions) {
  try {
    await webpush.sendNotification(
      {
        endpoint: sub.endpoint,
        keys: { p256dh: sub.p256dh, auth: sub.auth },
      },
      JSON.stringify(payload)
    )
  } catch (error: any) {
    if (error.statusCode === 410 || error.statusCode === 404) {
      await prisma.pushSubscription.delete({ where: { id: sub.id } })
    }
  }
}
```

التأثير: بدون هذا الإصلاح، المستخدمون على أجهزة متعددة لا يستقبلون الإشعارات.

المخفي Toast توسيع طابور 2.3

المشكلة: `hiddenToastQueueRef` في `FloatingBell.tsx` عن التبويب `5` إشعارات. إذا كان المستخدم بعيداً عن التبويب `5` إشعارات. مقيّد بـ `5` إشعارات. فترة طويلة، تُفقد الإشعارات الزائدة.

الحل: زيادة الحد إلى `20`، أو جعله ديناميكياً بناءً على الوقت المنقضي.

الموقع: `src/components/ui/FloatingBell.tsx` — البحث عن `hiddenToastQueueRef`

التغيير:

```
// قبل
const hiddenToastQueueRef = useRef<any[]>([])
const MAX_HIDDEN_TOASTS = 5

// بعد
const hiddenToastQueueRef = useRef<any[]>([])
const MAX_HIDDEN_TOASTS = 20
```

التأثير: المستخدمون الذين يعودون بعد غياب طويل قد يفقدون إشعارات.

المرحلة 3: تحسين الأمان والمراقبة

logging الهدف: سد الثغرات الأمنية وتحسين
نوع التغيير: تعديل + إضافة

الخطوة: متوسطة

3.1 (Real-time) تشفير الرسالة في البث المباشر

Supabase Realtime ولكن تُبث نصاً عادياً عبر `encryptMessage` (المشكلة: الرسائل تُخزن مشفرة في قاعدة البيانات WebSocket. أي شخص لديه صلاحية الوصول إلى قناة المستخدم يمكنه قراءة محتوى الرسالة.

لتشفير الرسالة قبل البث المباشر أيضاً `MessageService.sendMessageWithSideEffects` **الحل:** تعديل

`sendMessageWithSideEffects` دالة — `src/services/chat/MessageService.ts` **الموقع:**

التغيير:

```
// قبل البث - تشفير الرسالة للبث
const broadcastPayload = {
  ...message,
  body: encryptMessage(message.body), // إضافة ←
}

// العادي message المشفر بدلاً من payload البث باستخدام
broadcastEvent(senderChannel, "new-message", broadcastPayload)
broadcastEvent(receiverChannel, "new-message", broadcastPayload)
```

عند استقبال رسالة جديدة، فك التشفير — `ChatPage.tsx` ثم في

```
// "new-message" عند استقبال حدث
const handler = (payload) => {
  const decryptedMessage = {
    ...payload,
    body: decryptMessage(payload.body),
  }
  // باقي المعالجة ...
}
```

والمفتاح موجود في `src/lib/crypto.ts` مستوردتان من `encryptMessage` و `decryptMessage` **ملاحظة:** دالتا أو استخدام آلية مختلفة client-side الخادم فقط. يجب توفير مفتاح آخر للـ

في المتصفح. ولكن هذا تغيير كبير. الإصلاح `crypto.subtle` **بديل أكثر أماناً:** إنشاء زوج مفاتيح لكل محادثة، أو استخدام بنفس مفتاح الخادم (مع العلم أن المفتاح سيكون في الكود المصدّر للمتصفح) payload الأبسط هو تشفير الـ

WebSocket **التأثير:** البيانات الحساسة (الرسائل) تنتقل نصاً عادياً عبر

3.2 الصامته Catch Blocks إصلاح

تبتلع الأخطاء دون تسجيل أو إشعار `MessageService.ts` و `FloatingBell.tsx` في `try/catch` **المشكلة:** بعض المستخدم. هذا يجعل تصحيح الأخطاء صعباً وقد يخفي مشاكل حقيقية

في **catch** الموقع: البحث عن

- `src/components/ui/FloatingBell.tsx`
- `src/services/chat/MessageService.ts`
- `src/lib/supabaseRealtime.ts`

صامت، مع إعادة رمي الخطأ إذا كان مناسباً **catch** block في كل `console.error` **الحل**: إضافة

مثال للتغيير:

```
// قبل
try {
  // ...
} catch {
  // صامت
}

// بعد
try {
  // ...
} catch (error) {
  console.error("[FloatingBell] الإشعار في معالجة الإشعار:", error)
  // للمستخدم إذا كان خطأ خرج Toast إظهار
  showToast("error", "حدث خطأ في الإشعارات")
}
```

التأثير: الأخطاء تُبتلع حالياً دون أي تسجيل

3.3 Push Subscriptions تحسين تنظيف

الأخطاء الأخرى. HTTP 410 (Gone) يحذف الاشتراكات منتهية الصلاحية فقط عند استقبال `sendPushToUsers` **المشكلة**: (403, 400, 404) لا تؤدي إلى حذف الاشتراك، مما يسبب تراكم اشتراكات غير صالحة

الموقع: `src/lib/pushNotifications.ts`

التغيير:

```
// قبل
if (error.statusCode === 410) {
  await prisma.pushSubscription.delete({ where: { id: sub.id } })
}

// بعد
if ([410, 404, 400, 403].includes(error.statusCode)) {
  await prisma.pushSubscription.delete({ where: { id: sub.id } })
  console.warn(`[Push] تم حذف اشتراك غير صالح ${sub.endpoint}`)
}
```

التأثير: تراكم الاشتراكات المنتهية يستهلك مساحة في قاعدة البيانات ويسبب محاولات إرسال فاشلة.

المرحلة 4: تحسين الأداء وتجربة المستخدم

UX الهدف: تسريع التطبيق وتحسين
(Additive) نوع التغيير: إضافي
الخطورة: منخفضة

4.1 (IndexedDB) إضافة تخزين مؤقت للرسائل

عند العودة إلى الصفحة، يُعاد الطلب من جديد. API كل المحادثات والرسائل تُحمل من `/chat` المشكلة: عند تحميل صفحة Cache لا يوجد.

للتخزين الدائم للرسائل. يمكن البدء بحل بسيط باستخدام IndexedDB مؤقتاً (للبداية) أو `localStorage` الحل: استخدام `localStorage` للمحادثات الأخيرة.

الموقع: `src/app/chat/page.tsx` أو إنشاء ملف مساعد `src/lib/messageCache.ts`

مخطط بسيط للمرحلة الأولى:

```
// src/lib/messageCache.ts
const CACHE_KEY = "chat-cache"
const CACHE_TTL = 5 * 60 * 1000 // 5 دقائق

export interface MessageCache {
  conversations: Conversation[]
  messages: Record<string, Message[]>
  timestamp: number
}

export const getMessageCache = (): MessageCache | null => {
  try {
    const raw = localStorage.getItem(CACHE_KEY)
    if (!raw) return null
    const cache: MessageCache = JSON.parse(raw)
    if (Date.now() - cache.timestamp > CACHE_TTL) {
      localStorage.removeItem(CACHE_KEY)
      return null
    }
    return cache
  } catch {
    return null
  }
}

export const setMessageCache = (data: Partial<MessageCache>) => {
  try {
    const existing = getMessageCache() ?? { conversations: [], messages: {}, timestamp: Date.now() }
```

```

    localStorage.setItem(CACHE_KEY, JSON.stringify({ ...existing, ...data,
timestamp: Date.now() })))
  } catch {
    // localStorage ممتلئاً قد يكون
  }
}

```

API التأثير: كل تحميل صفحة يُعيد جلب جميع البيانات من

4.2 typing_stop تحسين توقيت

قد تكون (سابقاً ms قُلِّصت من 5000 ms يستخدم مهلة 3000 في ChatArea.tsx في resetTypingStop **المشكلة:** 3000ms لا تزال طويلة بعض الشيء).

الحل: ms. تقليص المهلة إلى 2000

الموقع: src/components/chat/ChatArea.tsx

التغيير:

```

// قبل
const TYPING_STOP_TIMEOUT = 3000

// بعد
const TYPING_STOP_TIMEOUT = 2000

```

التأثير: المستخدم قد يرى "يكتب الآن..." لمدة 3 ثوان بعد توقف الطرف الآخر عن الكتابة.

4.3 عند فشل إنشاء الإشعار Retry إضافة

يتم prisma.notification.create إذا فشل، MessageService.sendMessageWithSideEffects **المشكلة:** في ولا يتم إعادة المحاولة (صامت catch) ابتلاع الخطأ.

بسيط (محاولتان) مع تسجيل الفشل retry **الحل:** إضافة نظام.

الموقع: src/services/chat/MessageService.ts

التغيير:

```

// إضافة دالة مساعدة
async function createNotificationWithRetry(data: any, retries = 2): Promise<void>
{
  for (let i = 0; i < retries; i++) {
    try {
      await prisma.notification.create({ data })
      return
    } catch (error) {

```

```
    if (i === retries - 1) {
      console.error("[Notification] فشل إنشاء الإشعار بعد", retries, "محاولات",
error)
    } else {
      await new Promise((r) => setTimeout(r, 1000))
    }
  }
}
```

التأثير: فشل عابر في قاعدة البيانات يؤدي إلى فقدان الإشعار دون أي إشعار للنظام.

ملخص التغييرات حسب الملف

الملف	المرحلة	التغيير
public/sw.js	1	Service Worker جديد — إنشاء
src/app/api/push/subscribe/route.ts	1	Push للتسجيل في API Route — جديد
src/components/ui/FloatingBell.tsx	1	"إضافة زر "تفعيل الإشعارات
src/app/layout.tsx	1	إضافة trackPresence() + beforeunload ل lastSeenAt
src/app/api/user/ping/route.ts	1	lastSeenAt لتحديث API — جديد
src/app/chat/page.tsx	1	Layout منقول إلى trackPresence إزالة
src/app/api/chat/send/route.ts	2	idempotencyKey تعزيز
src/services/chat/MessageService.ts	2	تلقائي idempotencyKey إنشاء
src/lib/pushNotifications.ts	2	cleanup تحسين + findUnique بدل findMany
src/components/ui/FloatingBell.tsx	2	زيادة MAX_HIDDEN_TOASTS 20 إلى 5
src/services/chat/MessageService.ts	3	تشفير الرسالة في البث المباشر
src/components/ui/FloatingBell.tsx	3	الصامته catch blocks إصلاح
src/app/chat/page.tsx	3	فك تشفير الرسالة عند استقبال البث
src/lib/messageCache.ts	4	جديد — نظام تخزين مؤقت للرسائل
src/app/chat/page.tsx	4	messageCache استخدام
src/components/chat/ChatArea.tsx	4	ms إلى 2000 TYPING_STOP_TIMEOUT تقليص

جدول الأولويات المقترح

المخاطر	المدة المتوقعة	المرحلة	الأولوية
منخفضة جداً — إضافات فقط	ساعات 2-3	المرحلة 1 (الإصلاحات الحرجة)	الأولى 🏆
متوسطة — تعديل كود موجود	ساعة 1-2	المرحلة 2 (سلامة البيانات)	الثانية 🥈
متوسطة — تعديل تشفير	ساعات 2-3	المرحلة 3 (الأمان والمراقبة)	الثالثة 🥉
منخفضة — إضافات بشكل رئيسي	ساعة 1-2	المرحلة 4 (الأداء)	4

كل (Git commit أو save). **ملاحظة مهمة:** قبل البدء بأي مرحلة، يُفضّل أخذ نسخة احتياطية من الملفات المتأثرة. مرحلة مصممة لتكون مستقلة وقابلة للاختبار بشكل منفصل.